Laboratoire 2

Apprendre à reconnaître des objets

LOG635 – Introduction aux systèmes | ÉTS | 15 % de la note finale

Objectif et jeu de données

- Concevoir et comparer plusieurs modèles d'apprentissage automatique
- Reconnaissance de marqueurs ArUco (proches des codes QR)
- EnsembleDeDonnéesA : 16 symboles, plusieurs photos par symbole
- Images créées par les étudiants des sessions précédentes

Vue d'ensemble du pipeline

1. EnsembleB
(≥ 8 symboles)

2. Prétraitement
(taille, ROI)

3. Altérations
(optionnel)

4. Caractéristiques
(pixels de la ROI)

5. Réseau de neurones

6. Évaluation
(CV ou hold-out)

7. Comparaison
SVM, k-NN, RN sklearn

8. Scalabilité
100 %, 80 %, 30 %

1. Préparation : l'EnsembleB

- Sous-ensemble de l'EnsembleA avec au moins 8 symboles distincts
- Plusieurs exemples par symbole
- Justification obligatoire du choix des symboles
- Moins de classes : tâche plus simple, entraînement plus rapide
- Plus de classes : meilleure généralisation, mais plus difficile

2. Prétraitement des images

- Réduction de taille (ex. 320×140 → 160×70) : optionnelle, mais recommandée
- Extraction de la ROI : isoler la zone du marqueur
- Manuelle, ou automatique avec OpenCV (détection de formes, contours)
- Pourquoi réduire : moins de caractéristiques, entraînement plus rapide

3. Altérations d'images (optionnel)

- Flou : atténue les détails, aide la détection de formes
- Niveaux de gris : réduit la complexité et le nombre de primitives
- Augmentation de données : rotations, translations, distorsions, bruit
- Toute altération doit être détaillée clairement dans le rapport

4. Extraction des caractéristiques

- Base : les pixels de la région d'intérêt
- Optionnel : nombre de formes dans l'image
- Optionnel : excentricité de la plus grande forme
- Normalisation recommandée (MinMaxScaler ou StandardScaler)

5. Réseau de neurones from scratch

- Au moins une couche cachée
- Aucune librairie prête à l'emploi (NumPy uniquement)
- Implémentation vectorisée : aucune boucle for
- Propagation avant (1,5 pt) et propagation arrière (1,5 pt)
- À décrire : type de réseau, couches cachées, activation, hyperparamètres

6. Évaluation du modèle

- Évaluation sur l'EnsembleB
- Option 1 : validation croisée à k échantillons (k = 3 ou 5)
- Option 2 : hold-out (entraînement / validation / test)
- Justifier la méthode et les hyperparamètres choisis

7. Comparaison avec scikit-learn

- Trois modèles : SVM, k-NN et RN
- Comparaison avec votre réseau
- Mêmes données, même méthode d'évaluation
- Normaliser les caractéristiques pour les deux réseaux

8. Tests de scalabilité

- Réduire les données d'entraînement à 80 % puis à 30 %
- Appliquer à tous les modèles
- Observer l'impact sur les performances
- Quel modèle résiste le mieux à moins de données?

9. Résultats à présenter

Élément	Format	Pts
Précision, rappel, F-mesure	Tableau	0,5
Scalabilité (80 % / 30 %)	Tableau ou graphe	0,5
Temps d'entraînement	Tableau ou graphe	0,5
Temps de prédiction	Tableau ou graphe	0,5
Interprétation et conclusion	Texte	0,5

Remise et barème

Un seul fichier .zip

- Images prétraitées, classées par étiquette
- Notebook Jupyter commenté (1er commentaire : absolute path du script sur le robot)
- Court rapport avec les résultats

Barème (sur 15)

- Ensemble de données : 2
- Réseau de neurones : 8
- Rapport : 5

Pénalités possibles (0 à 10 % chacune) : lisibilité du code, non-respect de la remise, qualité du rapport